



Databases and its Applications

Relational Modelling

Pooja T S
Computer Applications

Databases and its Applications

Relational Modelling

SQL: Joins – Inner, Outer, Cross

Pooja T S

Computer Applications



Databases and its Applications

Introduction to Joins

- ▶ In relational databases, data is often stored across multiple related tables.
- ▶ The **JOIN** clause is used to combine rows from two or more tables based on a related column between them.
- ▶ In PostgreSQL, joins are evaluated by matching values of common keys.
- ▶ Joins enable meaningful relationships like student-department, order-customer, etc.



Databases and its Applications

Types of Joins in SQL

- ▶ SQL supports several join types:
 - **INNER JOIN** – returns matching rows only.
 - **LEFT OUTER JOIN** – all rows from left table, matching from right.
 - **RIGHT OUTER JOIN** – all rows from right table, matching from left.
 - **FULL OUTER JOIN** – all rows when match exists in either table.
 - **CROSS JOIN** – Cartesian product of both tables.
- ▶ PostgreSQL also supports **NATURAL JOIN** and **USING()** for convenience.



Databases and its Applications

Setup for Demonstration

- ▶ We use two tables:
 - `department(deptid, deptname)`
 - `student(sid, sname, deptid, cgpa)`
- ▶ Each student belongs to one department via the deptid foreign key.



Databases and its Applications

INNER JOIN – Concept

- ▶ The **INNER JOIN** returns rows that have matching values in both tables.
- ▶ Rows without a match in either table are excluded.
- ▶ Most frequently used join type in SQL queries.



Databases and its Applications

INNER JOIN – Syntax

- ▶ `SELECT columns`
`FROM table1`
`INNER JOIN table2`
`ON table1.column = table2.column;`
- ▶ The ON clause specifies the join condition.



Databases and its Applications

INNER JOIN – Example

► Command:

```
SELECT s.sname, d.deptname, s.cgpa
FROM student s
INNER JOIN department d
ON s.deptid = d.deptid;
```

► Output (sample):

sname	deptname	cgpa
Ravi	CSE	8.40
Kavya	ECE	9.10
Anita	BCA	7.80

(3 rows)



Databases and its Applications

LEFT OUTER JOIN – Concept

- ▶ The **LEFT JOIN** returns all rows from the left table and matched rows from the right.
- ▶ If no match is found, columns from the right table are shown as NULL.



Databases and its Applications

LEFT OUTER JOIN – Example

► Command:

```
SELECT s.sname, d.deptname  
FROM student s  
LEFT JOIN department d  
ON s.deptid = d.deptid;
```

► Output (sample):

sname	deptname
Ravi	CSE
Kavya	ECE
Manoj	NULL

(3 rows)

► Meaning: “Manoj” has no matching department.



Databases and its Applications

RIGHT OUTER JOIN – Concept

- ▶ The **RIGHT JOIN** returns all rows from the right table and only matching rows from the left.
- ▶ It is effectively the mirror of a LEFT JOIN.



Databases and its Applications

RIGHT OUTER JOIN – Example

► **Command:**

```
SELECT d.deptname, s.sname  
FROM student s  
RIGHT JOIN department d  
ON s.deptid = d.deptid;
```

► **Output (sample):**

deptname	sname
CSE	Ravi
ECE	Kavya
IT	NULL

(3 rows)



Databases and its Applications

FULL OUTER JOIN – Concept

- ▶ The **FULL OUTER JOIN** combines results of both LEFT and RIGHT joins.
- ▶ All records from both tables appear, with NULLs where there is no match.



Databases and its Applications

FULL OUTER JOIN – Example

► Command:

```
SELECT s.sname, d.deptname  
FROM student s  
FULL OUTER JOIN department d  
ON s.deptid = d.deptid;
```

► Output (sample):

sname	deptname
Ravi	CSE
Kavya	ECE
Manoj	NULL
NULL	IT

(4 rows)



Databases and its Applications

CROSS JOIN – Concept

- ▶ The **CROSS JOIN** produces a Cartesian product of both tables.
- ▶ Every row in the first table is combined with every row in the second.
- ▶ Use only when required — can explode results quickly.



Databases and its Applications

CROSS JOIN – Example

► **Command:**

```
SELECT s.sname, d.deptname  
FROM student s  
CROSS JOIN department d;
```

► **Output (sample):**

sname	deptname
Ravi	CSE
Ravi	ECE
Kavya	CSE
Kavya	ECE

... total 9 rows for 3x3 combination ...



- ▶ PostgreSQL allows simplified join syntax:
 - `USING (common_column)` – shorthand for equality on same-named column.
 - `NATURAL JOIN` – automatically joins on all common column names.
- ▶ Example: `SELECT * FROM student JOIN department USING (deptid);`



Databases and its Applications

Self Join – Concept

- ▶ A **self-join** is when a table is joined to itself.
- ▶ Useful for hierarchical or comparative data (e.g., employee–manager relationships).



Databases and its Applications

Self Join – Example

- ▶ Consider a table employee(empid, name, managerid).
- ▶ Command:

```
SELECT e.name AS employee, m.name AS manager  
FROM employee e  
LEFT JOIN employee m  
ON e.managerid = m.empid;
```
- ▶ Output (sample):

employee	manager
Ravi	Kavya
Kavya	NULL

(2 rows)



Databases and its Applications

Performance Notes on Joins

- ▶ PostgreSQL uses cost-based optimization to pick best join algorithm:
 - **Nested Loop Join** – for small data sets or indexed keys.
 - **Hash Join** – for equality joins on large tables.
 - **Merge Join** – when inputs are sorted.



Databases and its Applications

Best Practices for Joins

- ▶ Always define PRIMARY KEY and FOREIGN KEY.
- ▶ Use table aliases (s, d) for readability.
- ▶ Avoid ambiguous column names — qualify them with table alias.
- ▶ Index join columns for better performance.
- ▶ For large tables, verify execution plan with EXPLAIN.



PES
UNIVERSITY

CELEBRATING 50 YEARS

Pooja T S
Assistant Professor
Department of Computer Applications
poojats@pes.edu
080-26721983 Extn: 233